


# Course Name:-…..DATABASE MANAGEMENT SYSTEM(DBMS).....

## Course Code:-.....DSC202A.....

**Name:- Rashmi.A.C**

**Semester:- 3rd sem/2nd year**

**Year:- 2025-2029**

# DBMS – Continuous Evaluation 4 (CE-04)

# Building a Relational Snack Brand & Production Batch Management Dashboard.

Using python, SQLite and ipywidgets in Jupyter notebook.

## Abstract.

A snack food company currently tracks its production on spreadsheets, which leads to duplicate entries, orphaned records and inconsistent costs. This project replaces that system with a relational database application built with Python's sqlite3 module and an ipywidgets dashboard that runs inside Jupyter Notebook. The database stores three related entities: snack brands, production batches and packing items (such as foil pouch, seasoning mix and frying oil). The company's integrity rules are enforced by the database itself through foreign keys with ON DELETE CASCADE: deleting a brand removes all of its production batches and their packing items, and deleting a batch removes all of its packing items, while deleting a single packing item removes only that item. The dashboard provides an Insert Panel, a Delete Panel with dynamic dropdowns, input validation, and four real-time tabs showing a combined JOIN view and the three raw tables.

## Problem Statement and Objectives.

The business needs to keep track of three separate but related concepts: the snack brands owned by the company, the production batches manufactured under each brand, and the packing items costed into each batch. Management has mandated that deletes cascade automatically (brand → batches → packing items), that the interface runs smoothly in a single environment without screen-rendering crashes, and that invalid input and stale dropdown selections are prevented. The objectives of this project are:

- **Design a relational schema** with primary keys, a unique brand name, foreign keys and ON DELETE CASCADE.
- **Seed the database automatically** with sample data when it is empty.
- **Build an Insert Panel** that reuses an existing brand or creates a new one, and adds one batch and one packing item per submission.
- **Build a Delete Panel** with dynamic dropdowns to delete a brand, a batch or a single packing item.
- **Display four real-time tabs:** a combined JOIN view and the three raw tables.
- **Validate input** by rejecting blank fields and non-numeric or negative costs.
- **Keep the interface consistent:** clear the inputs and refresh all tabs after every action, and never leave a dropdown pointing at a deleted ID.

# 1. Aim and Introduction.

A snack food company tracks its production on chaotic spreadsheets, which leads to duplicated names, orphaned records and inconsistent costs. The aim of this project is to replace that system with a robust relational database application that tracks three related concepts:

- **Snack Brands** (product lines) owned by the company;
- **Production Batches** manufactured under each brand;
- **Packing Items** (for example foil pouch, seasoning mix, frying oil) that are costed into each batch.

Management has mandated strict integrity rules: deleting a brand must automatically remove all of its batches and their packing items, and deleting a batch must automatically remove its packing items. The application must run smoothly inside a single environment (Jupyter Notebook) without screen-rendering crashes, validate user input, and keep every view and dropdown in sync with the database after each action.

# 2. Tools and Technologies Used.

| Tool | Purpose |
|---|---|
| Python 3 (Anaconda) | Programming language used for the entire application. |
| SQLite (sqlite3 module) | Lightweight relational database stored in the file snack_company.db. |
| pandas | Converts SQL query results into tidy tables (DataFrames) for display. |
| ipywidgets | Builds the graphical interface (text boxes, buttons, dropdowns, tabs) inside the notebook, so no separate window is needed. |
| JupyterLab / Jupyter Notebook | Single development environment in which the code runs and the app is displayed. |

**Why ipywidgets instead of Tkinter?** Tkinter opens a separate desktop window, which is a well-known source of freezing and rendering problems when used from a notebook. ipywidgets draws the interface inside the notebook output, which satisfies the requirement of running smoothly in a single environment without rendering crashes.

**Why SQLite and ipywidgets?**

- **SQLite** ships with Python through the sqlite3 module and stores all data in a single file (snack_company.db). It supports full SQL, transactions, foreign keys and cascading deletes, so no separate database server needs to be installed.

- **ipywidgets** provides ready-made buttons, text boxes, dropdowns and tabs that render inside the notebook, so the whole project runs in one environment.
- **pandas** turns query results into tables, which makes the four tabs easy to display.
- **Parameterised queries** are used everywhere to prevent SQL injection, and each add or delete runs in one transaction so data is saved only if every step succeeds.

# 3. Database Design.

The database contains three tables connected in a parent–child chain: one brand has many production batches, and one production batch has many packing items.

**Relationship:** brands (1) → (many) production_batches (1) → (many) packing_items

## 3.1 Table structure.

Table 1: brands

| Column | Type | Constraint / Meaning |
|---|---|---|
| brand_id | INTEGER | Primary key, auto-increment |
| brand_name | TEXT | NOT NULL, UNIQUE – no duplicate brands |
| supervisor_email | TEXT | NOT NULL – contact of the production supervisor |

Table 2: production_batches

| Column | Type | Constraint / Meaning |
|---|---|---|
| batch_id | INTEGER | Primary key, auto-increment |
| brand_id | INTEGER | Foreign key → brands(brand_id), ON DELETE CASCADE |
| snack_name | TEXT | NOT NULL – e.g. Masala Potato Chips |
| manufacture_date | TEXT | NOT NULL – ISO date (YYYY-MM-DD) |

Table 3: packing_items

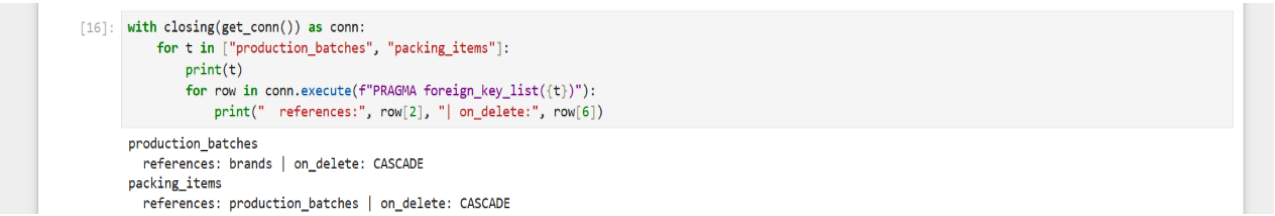
| Column | Type | Constraint / Meaning |
|---|---|---|
| item_id | INTEGER | Primary key, auto-increment |
| batch_id | INTEGER | Foreign key → production_batches(batch_id), ON DELETE CASCADE |
| item_description | TEXT | NOT NULL – e.g. Foil Pouch |
| cost_amount | REAL | NOT NULL, CHECK (cost_amount >= 0) |

## 3.2 Key database concepts.

- **Primary key:-** a unique ID that identifies each row.

- **Foreign key:-** a column in a child table that points to the primary key of its parent, preventing orphaned records.
- **ON DELETE CASCADE:-** when a parent row is deleted, the database automatically deletes its child rows. This is how the business rules are enforced.
- **PRAGMA foreign_keys = ON:-** SQLite ignores foreign keys by default, so this statement is executed on every new connection. Without it the cascade rules would silently not work.
- **UNIQUE and CHECK:-** UNIQUE on brand_name prevents duplicate brands; CHECK (cost_amount >= 0) rejects negative costs at the database level as a second line of defence.

The foreign-key definitions were verified directly from the database using PRAGMA foreign_key_list. The output below confirms that both child tables use ON DELETE CASCADE.



```
[16]: with closing(get_conn()) as conn:
          for t in ["production_batches", "packing_items"]:
              print(t)
              for row in conn.execute(f"PRAGMA foreign_key_list({t})"):
                  print("  references:", row[2], "| on_delete:", row[6])

production_batches
  references: brands | on_delete: CASCADE
packing_items
  references: production_batches | on_delete: CASCADE
```

**Figure 1:** PRAGMA foreign_key_list output: production_batches references brands, and packing_items references production_batches, both with on_delete: CASCADE.

# 3.3 Schema (SQL).

The three tables are created with the following SQL statements. The **PRAGMA foreign_keys = ON** statement is executed on every new connection so that SQLite enforces the foreign keys and the cascade rules.

```
PRAGMA foreign keys = ON;

CREATE TABLE IF NOT EXISTS brands (
    brand_id         INTEGER PRIMARY KEY AUTOINCREMENT,
    brand name       TEXT NOT NULL UNIQUE,
    supervisor_email TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS production_batches (
    batch_id         INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    brand_id         INTEGER NOT NULL,
    snack_name       TEXT NOT NULL,
    manufacture_date TEXT NOT NULL,
    FOREIGN KEY (brand_id) REFERENCES brands(brand_id)
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS packing items (
    item_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    batch_id         INTEGER NOT NULL,
    item_description TEXT NOT NULL,
    cost_amount      REAL NOT NULL CHECK (cost_amount >= 0),
    FOREIGN KEY (batch id) REFERENCES production batches(batch id)
        ON DELETE CASCADE
);
```

## 3.4 Cascading Delete Behaviour.

| Action | Automatic result |
|---|---|
| Delete a brand | All production batches of that brand are deleted, and then all packing items of those batches |
| Delete a production batch | All packing items of that batch are deleted; the brand remains |
| Delete a packing item | Only that item is removed; the batch and the brand remain |

# 4. Implementation.

The application is written as five notebook cells that are run in order. The complete source code is given in the Appendix.

### Cell 1 – Imports.

Imports sqlite3 (database), math (to reject non-finite numbers), closing (to always close connections), date (to stamp each batch with today's date), pandas (tables), ipywidgets (interface) and display/clear_output (to redraw only the changed output areas). The database file name is stored in DB_NAME.

## Cell 2 – Schema and automatic seeding.

get_conn() opens a connection and turns on foreign-key enforcement. create_tables() creates the three tables with primary keys, foreign keys and ON DELETE CASCADE. seed_if_empty() counts the rows in brands and, only if the table is empty, inserts two sample brands (Crunchy Munch and Cheesy Bites) with three batches and seven packing items. Because of this check, re-running the notebook never creates duplicate sample data.

## Cell 3 – Business logic.

- **add_record():** looks up the brand name (case-insensitive). If it exists its ID is reused; otherwise a new brand profile is created with an auto-generated supervisor email. It then inserts one production batch (dated today) and one packing item. All three inserts run in one transaction, so either everything is saved or nothing is.
- **delete_record():** deletes a row by ID using a parameterised query from the DELETE_SQL dictionary; the cascades are handled by the database.
- **parse_cost():** converts the cost text to a number and returns None for text, infinity or negative values.
- **JOIN_SQL:** joins production_batches, brands and packing_items to produce the combined view. A LEFT JOIN is used on packing_items so that a batch remains visible even if all its items have been deleted.
- **Parameterised queries :** used everywhere to prevent SQL injection.

## Cell 4 – Graphical user interface.

- **Insert Panel:** four text boxes (Brand, Snack, Item, Cost) and an Add Record button.
- **Delete Panel:** a Delete-type dropdown (Brand / Production Batch / Packing Item), a dynamic Select dropdown and a Delete Selected button.
- **Tabbed display:** a Tab widget containing four Output areas – Combined View, Brands, Production Batches and Packing Items. Using separate Output areas means only the changed tables are redrawn, which prevents rendering crashes.

- **refresh_all():** called after every add or delete; it reloads all four tabs and rebuilds the Select dropdown from the database.
- **load_targets():** builds the dropdown options from live data and always resets the selected value to the first remaining option (or None), so the dropdown can never point at a deleted ID.
- **on_add() / on_delete():** event handlers that validate input, perform the operation, clear the text boxes, refresh the display and show a green success or red error message.

### Cell 5 – Foreign-key verification.

A short check that prints the foreign-key definitions of the two child tables, used as evidence in Section 3.

## 5. Testing and Output Screenshots.

Every requirement of the assignment was tested in the running application. Each test is shown below with a screenshot and a short explanation.

### 5.1 Application on first run (auto-seeding).

When the notebook is run for the first time the database is empty, so seed_if_empty() fills it with sample data. The Insert Panel, Delete Panel and four tabs are displayed, and the Combined View shows 3 batches and 7 packing items for the brands Crunchy Munch and Cheesy Bites.

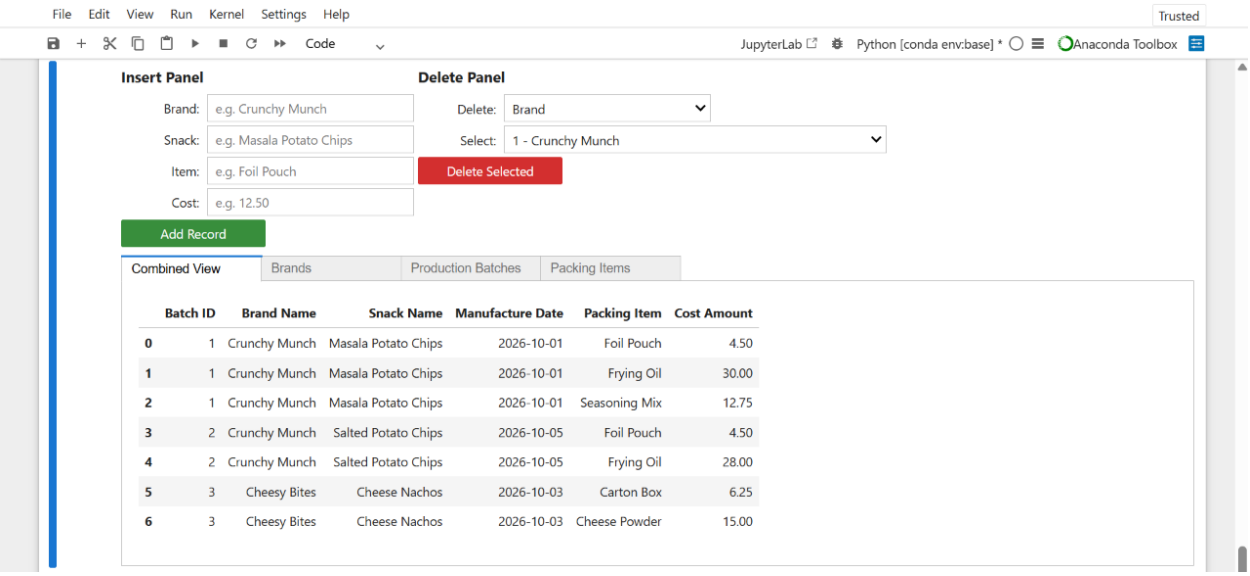



**Figure 2:** Application on first run – Insert Panel, Delete Panel, tabs and seeded data in the Combined View.

## 5.2 Tabbed display – four real-time views.

The Combined View (Figure 2 above) joins all three tables. The other three tabs show the raw tables. The following screenshots were taken at the end of testing, so they show the final database state (2 brands, 3 batches, 7 items).

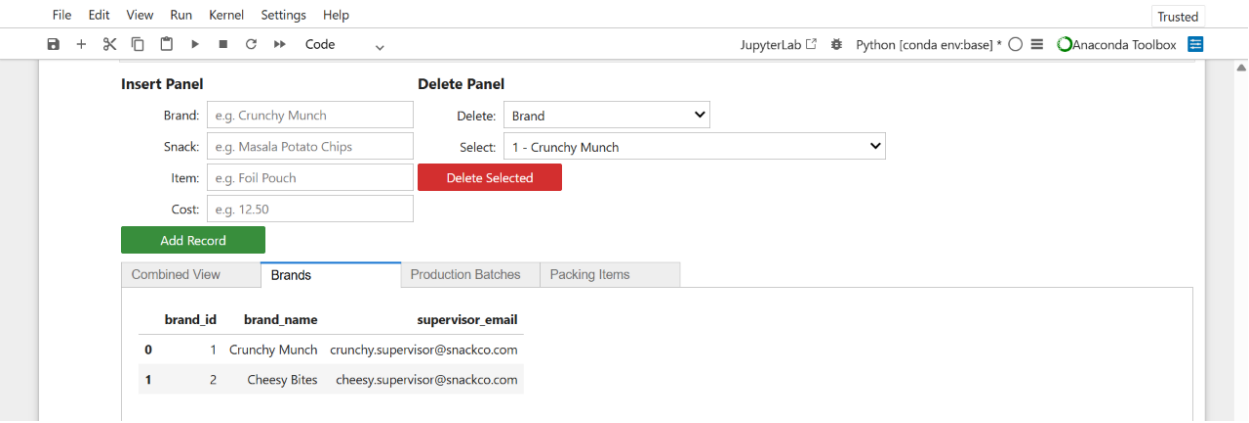



**Figure 3:** Tab 2 – Brands (raw table: brand_id, brand_name, supervisor_email).

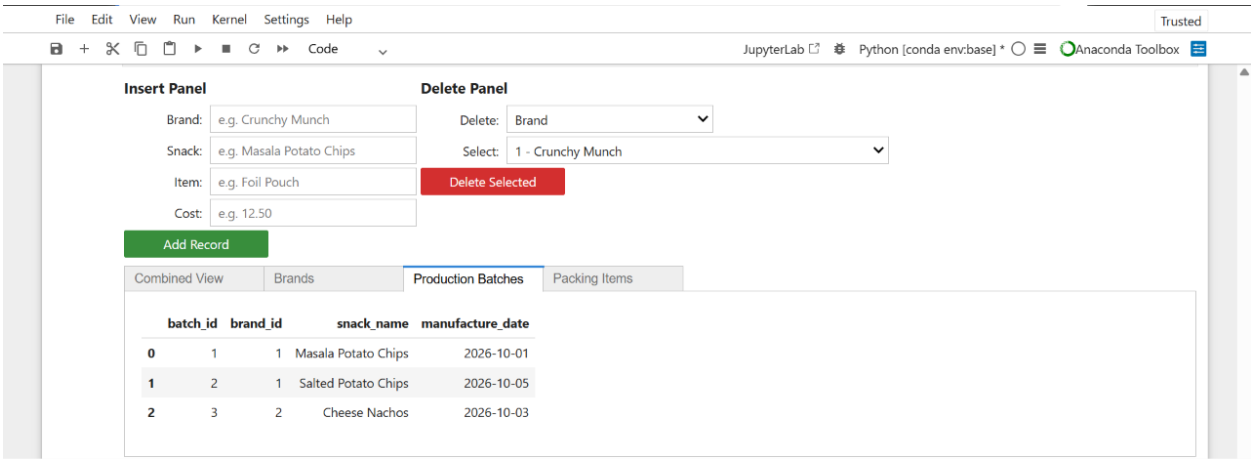



**Figure 4:** Tab 3 – Production Batches (raw table: batch_id, brand_id, snack_name, manufacture_date).

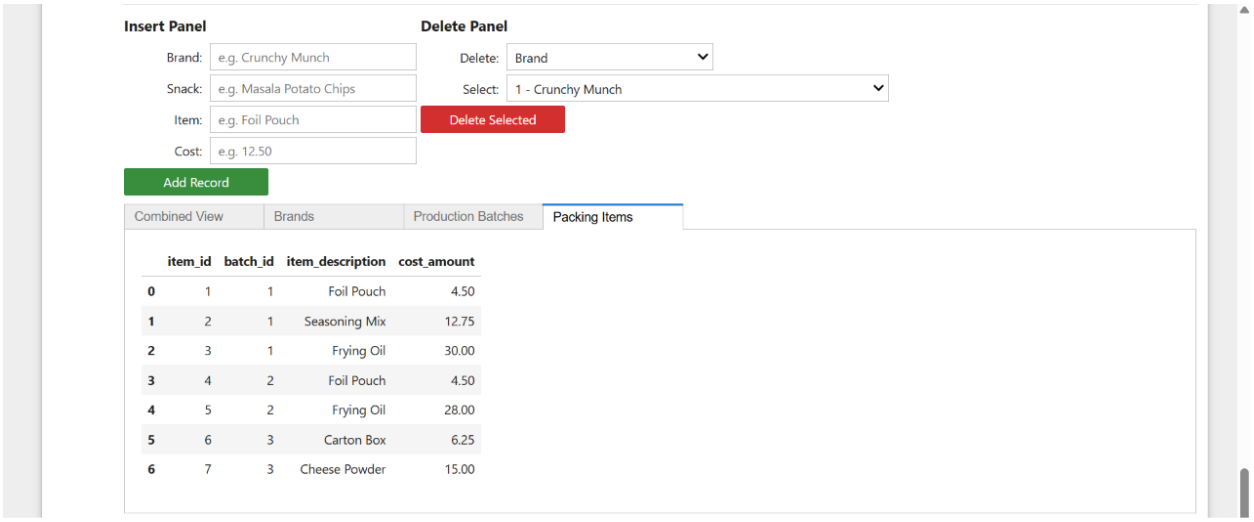



**Figure 5:** Tab 4 – Packing Items (raw table: item_id, batch_id, item_description, cost_amount).

## 5.3 Input validation.

The interface rejects blank fields and invalid costs. In each case a red message is shown and nothing is written to the database.

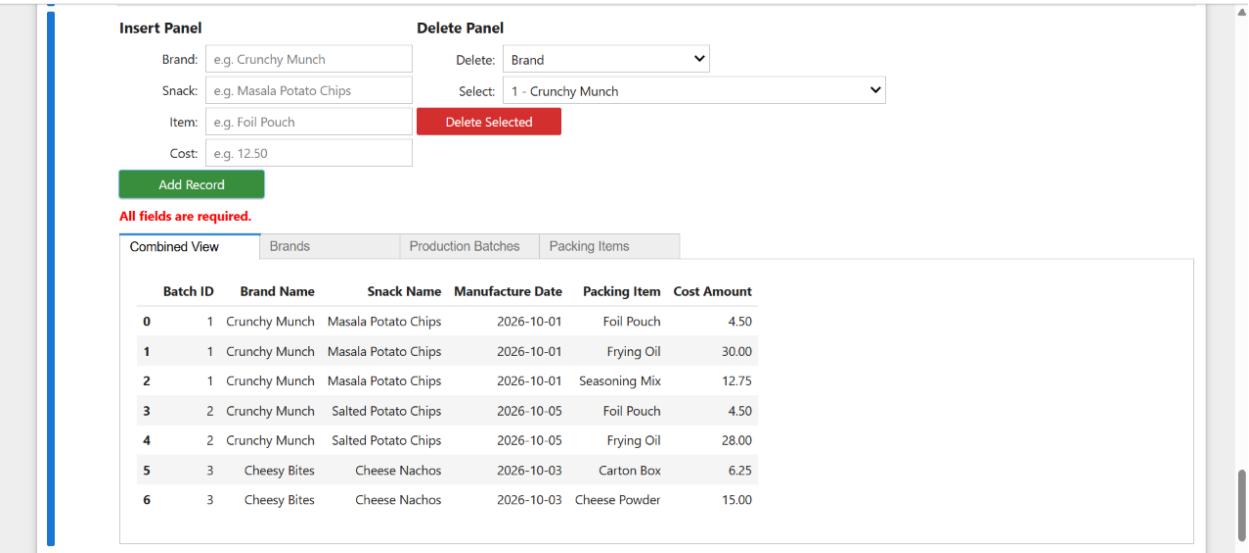



**Figure 6:** Validation – clicking Add Record with all fields empty shows "All fields are required."

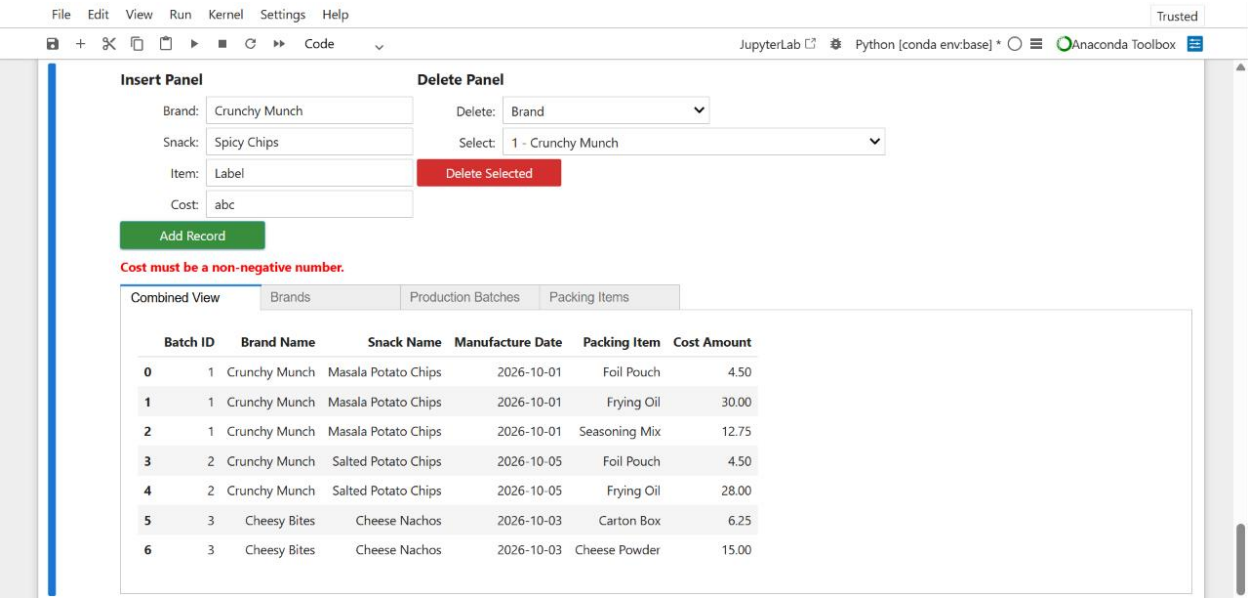



**Figure 7:** Validation – non-numeric cost ("abc") is rejected with "Cost must be a non-negative number."

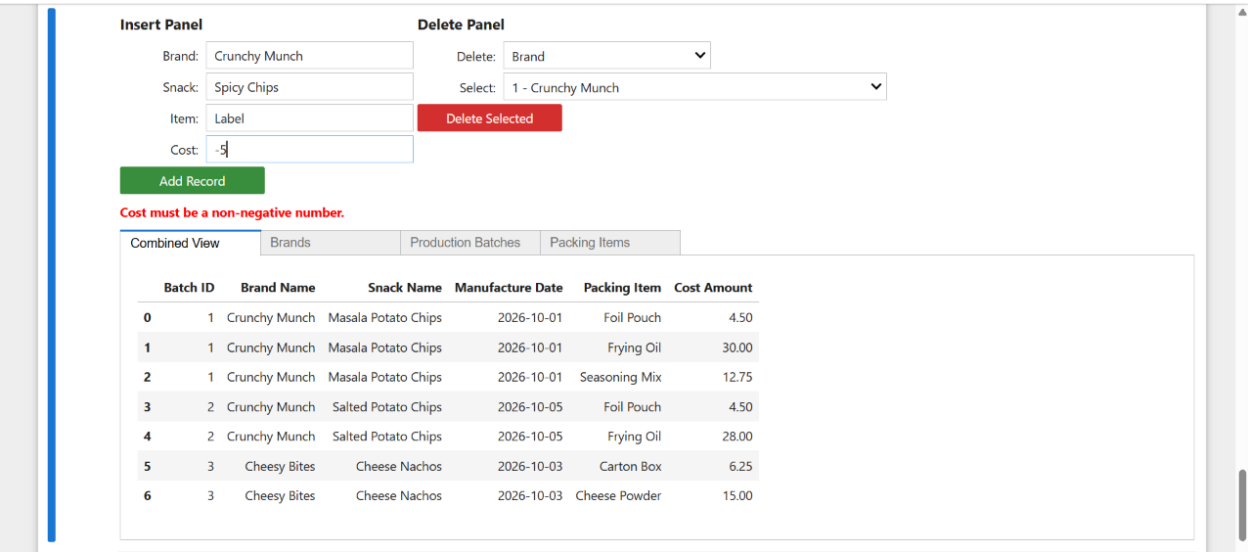



**Figure 8:** Validation – negative cost ("-5") is rejected with the same error message.

## 5.4 Adding a record for an existing brand.

The brand Crunchy Munch already exists (Brand ID 1). Adding Snack "Spicy Chips", Item "Label" and Cost 3.5 reuses the existing brand ID instead of creating a duplicate, and creates one new batch (Batch 6) with one packing item. The text boxes are cleared and the tables refresh instantly.

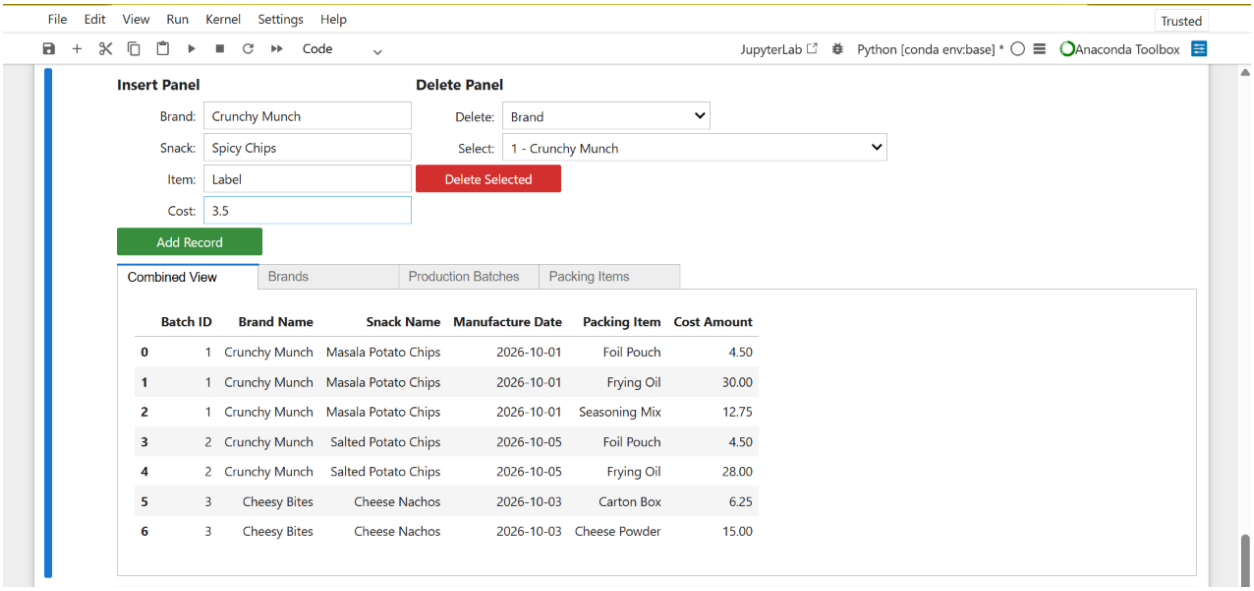



**Figure 9:** Valid input entered for an existing brand (Crunchy Munch, Spicy Chips, Label, 3.5).

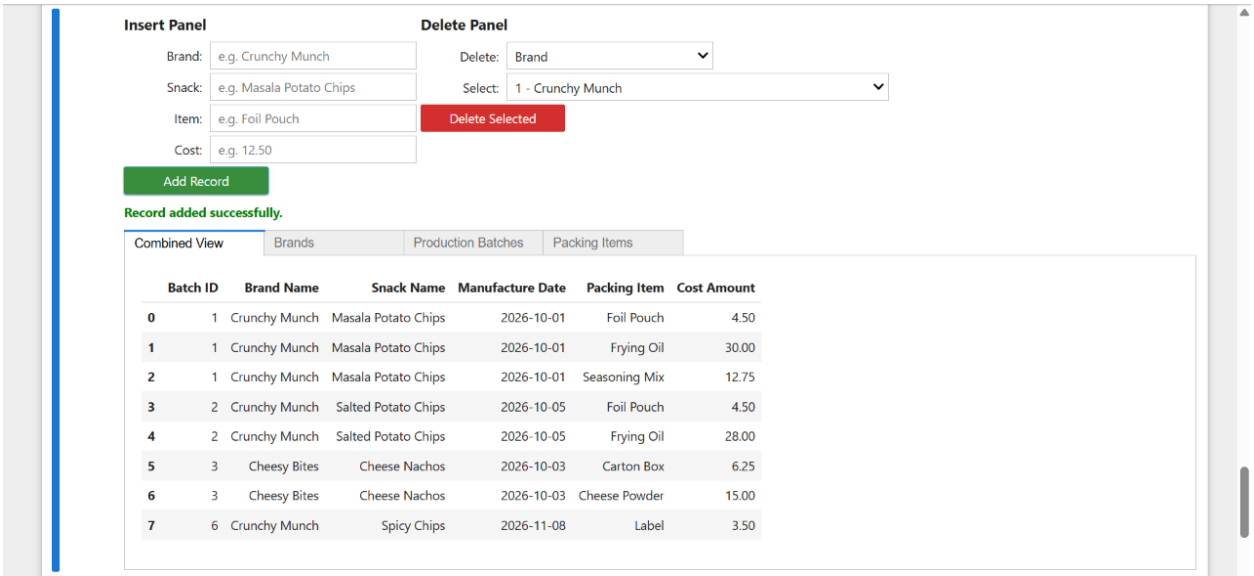



**Figure 10:** After clicking Add Record – success message, cleared input boxes and the new row (Batch 6) in the Combined View.

## 5.5 Adding a record for a new brand.

Entering a brand that does not exist (Tasty Crisps) automatically creates a new brand profile (Brand ID 3) with a generated supervisor email, followed by one new batch (Batch 7) and one packing item.

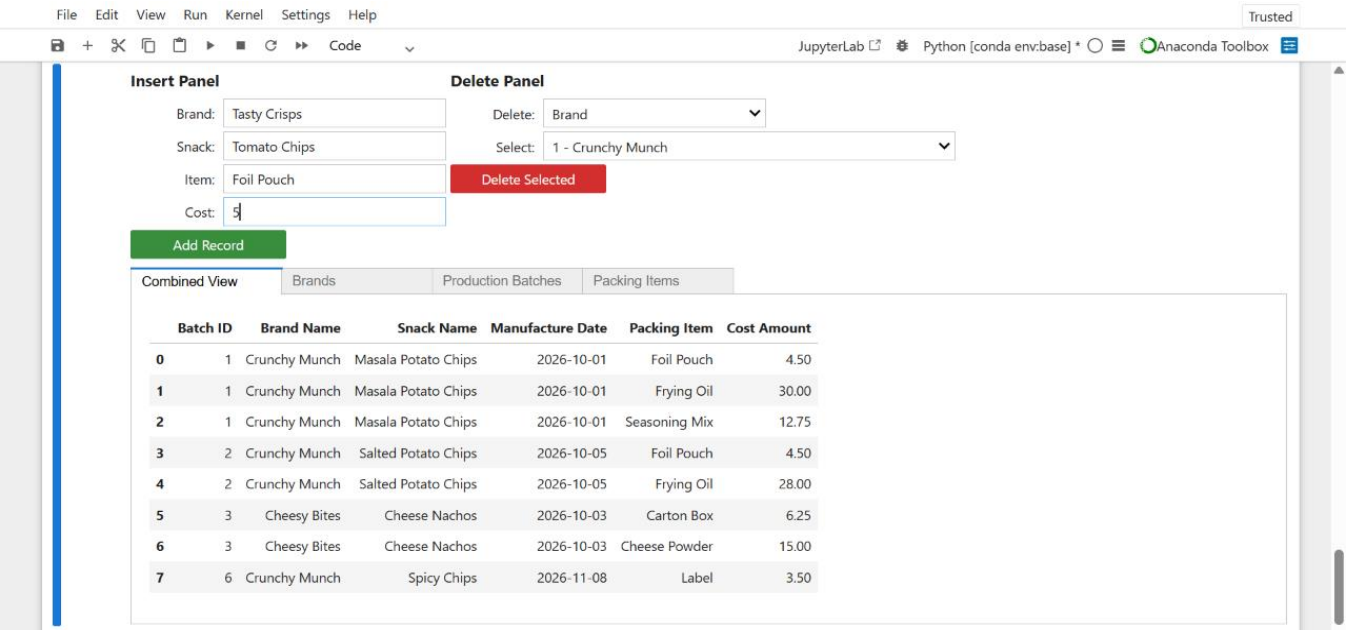



**Figure 11:** Valid input entered for a new brand (Tasty Crisps, Tomato Chips, Foil Pouch, 5).

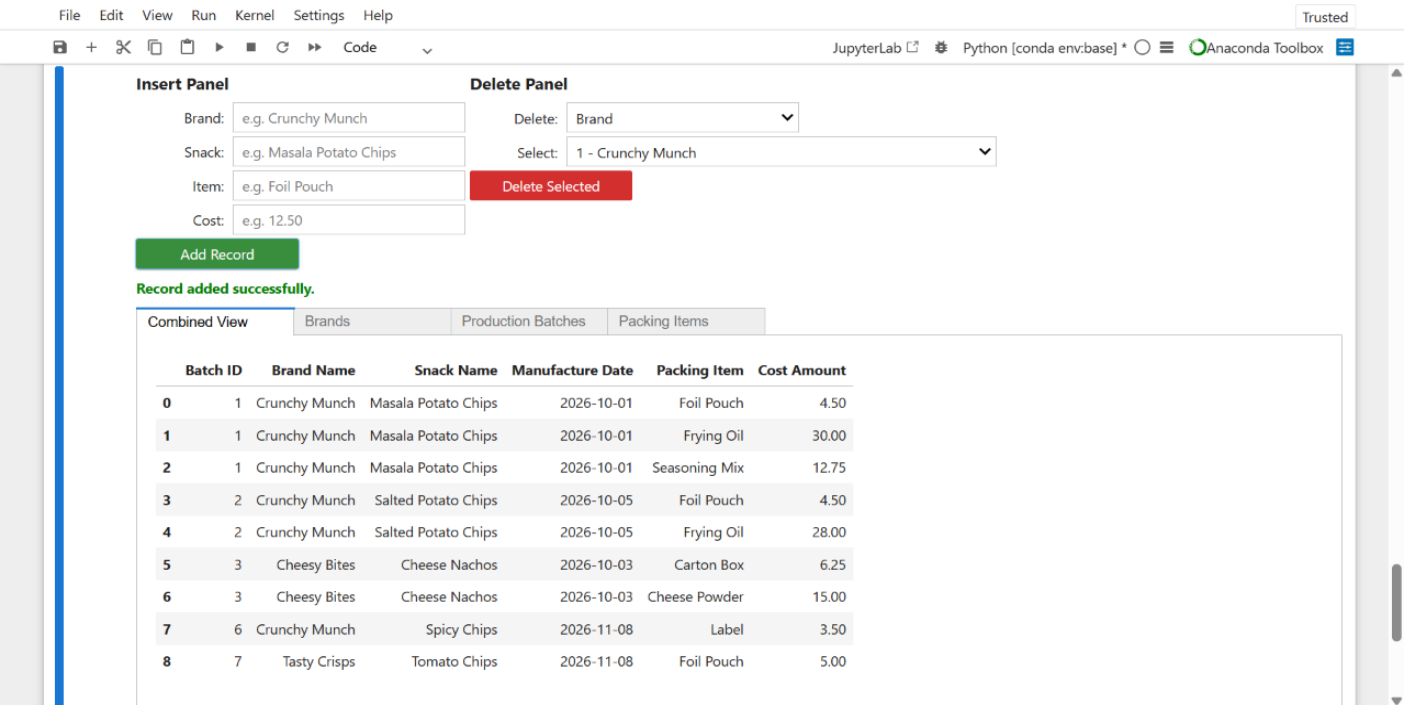



**Figure 12:** After clicking Add Record – the new brand's batch and item appear in the Combined View.

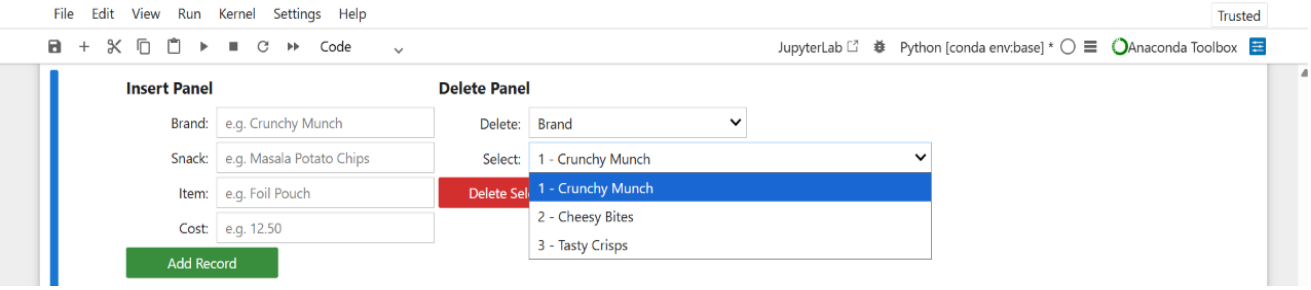



**Figure 13:** The Delete Panel dropdown is loaded dynamically from the database and now lists three brands, including Tasty Crisps.

## 5.6 Deleting a Packing Item (only that row is removed).

Delete type "Packing Item" was selected and Item 11 (Foil Pouch, Batch 7) was deleted. Only that row was removed: Batch 7 and the brand Tasty Crisps remain, and the Combined View shows empty values (None / NaN) for the packing item because of the LEFT JOIN.

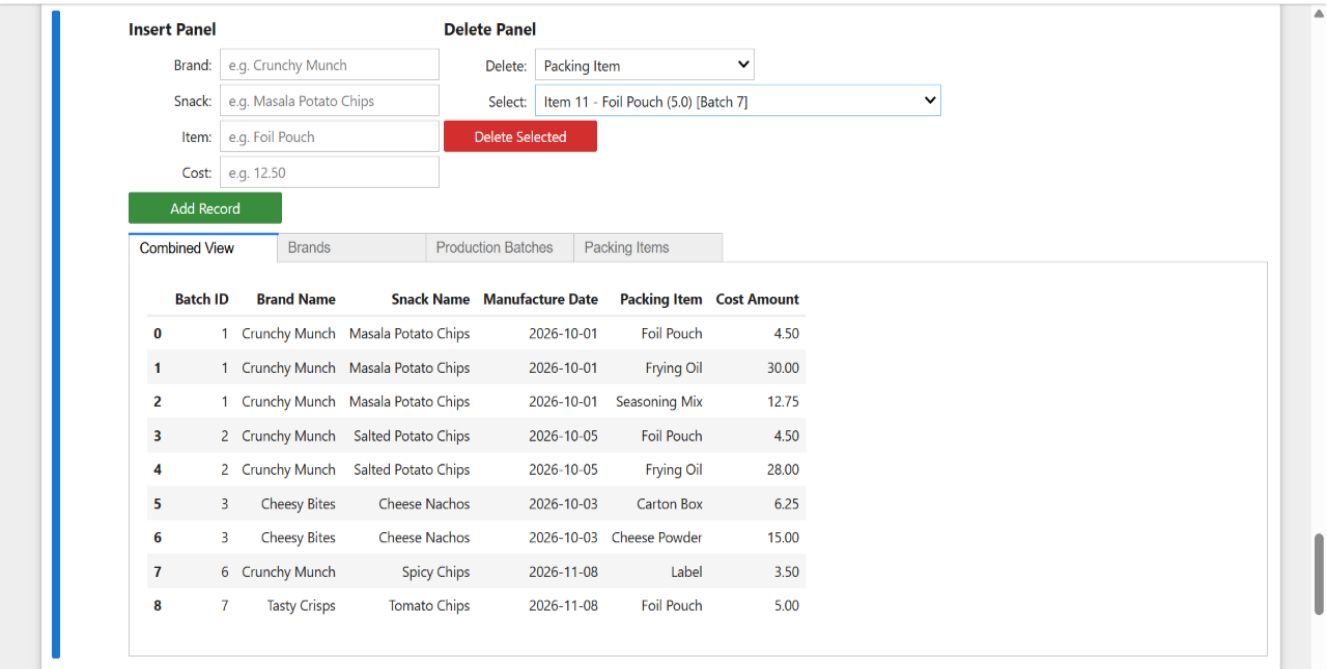



**Figure 14:** Before deletion – Item 11 (Foil Pouch, Batch 7) selected in the Delete Panel.

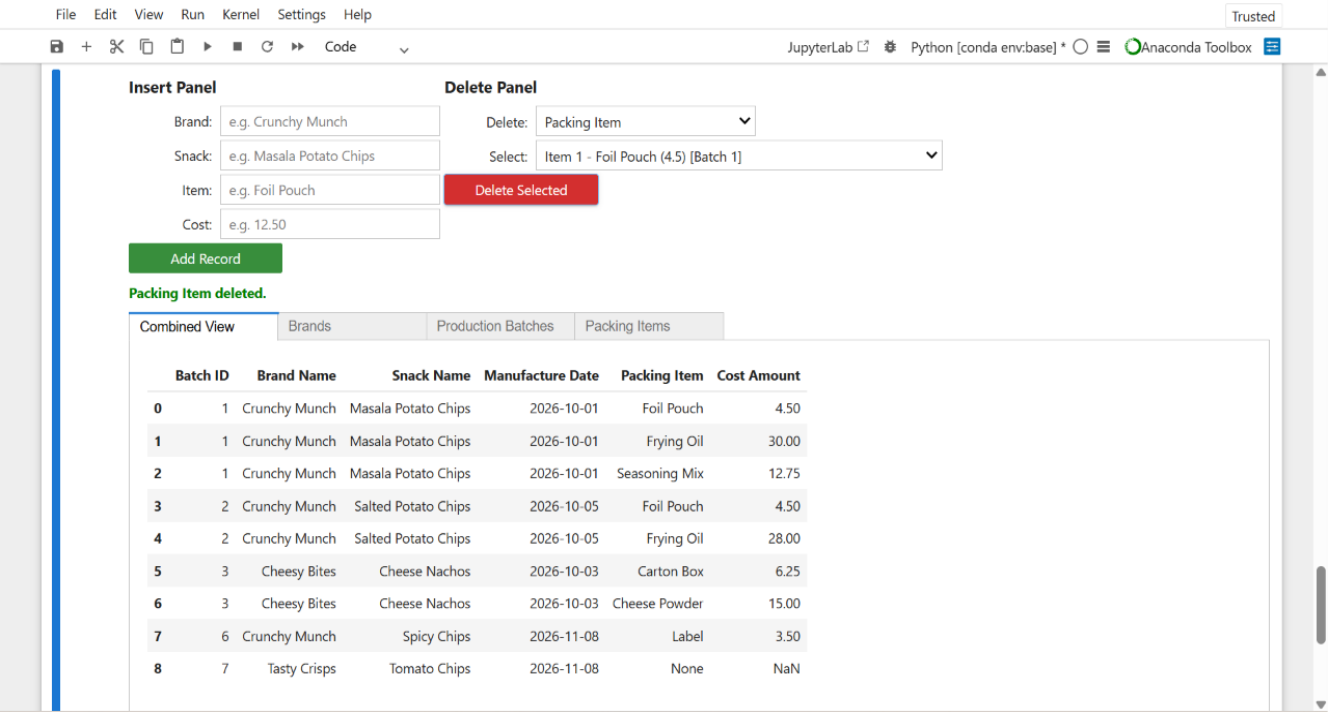



**Figure 15:** After deletion – "Packing Item deleted." The batch and brand remain; the packing item and cost are empty.

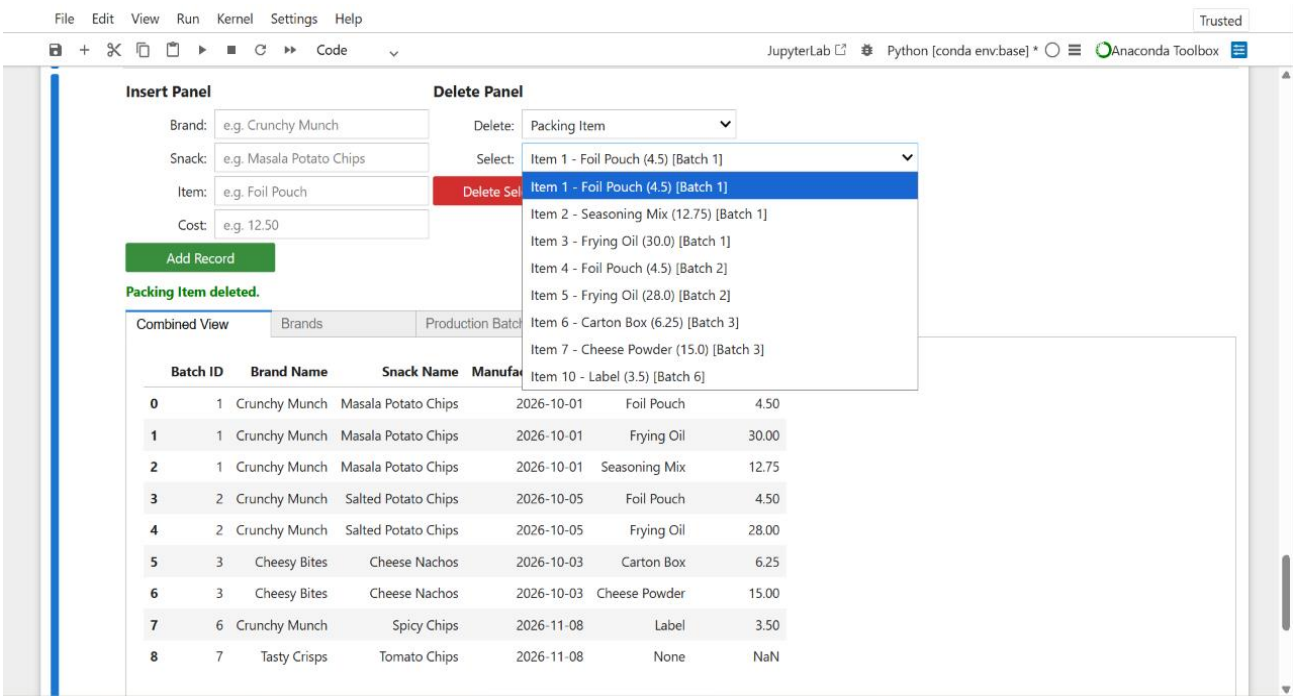



**Figure 16:** The Select dropdown after deletion no longer lists Item 11, and shows a valid remaining item.

## 5.7 Deleting a Production Batch (cascade to packing items).

Batch 6 (Crunchy Munch / Spicy Chips) was selected and deleted. Its packing item ("Label") was removed automatically by ON DELETE CASCADE, while the brand Crunchy Munch remained.

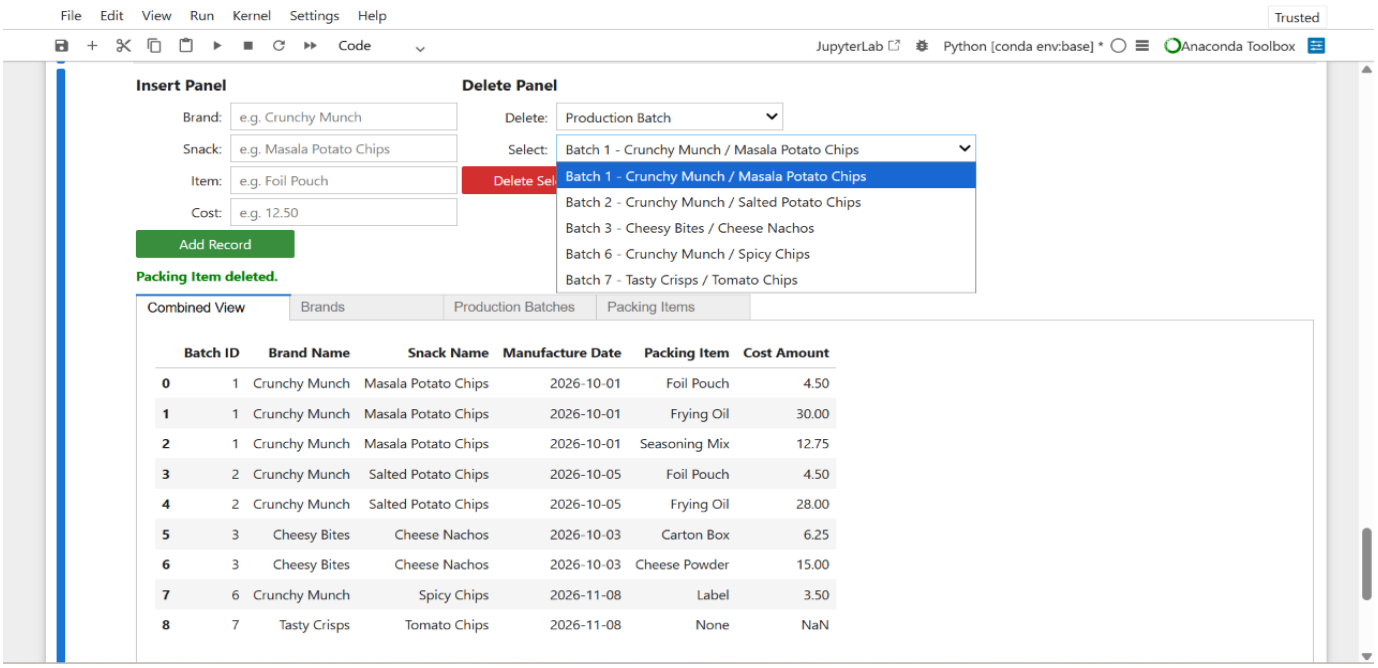
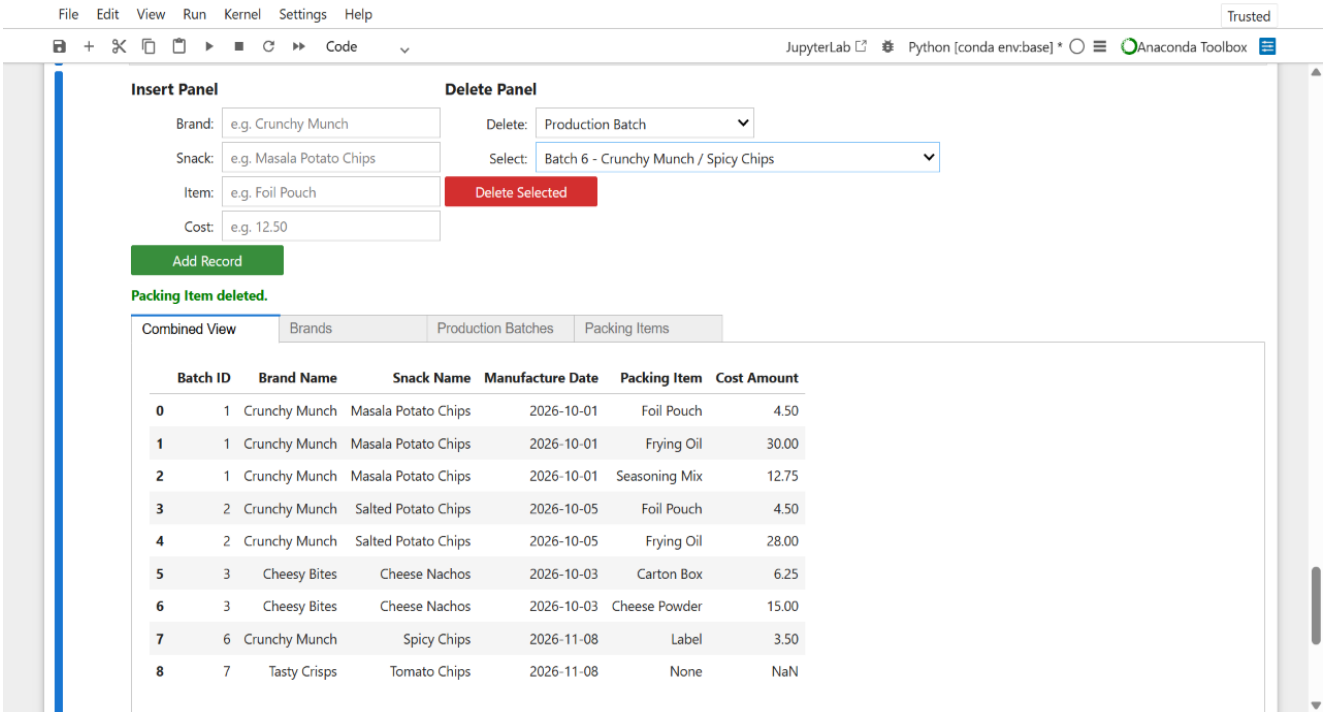
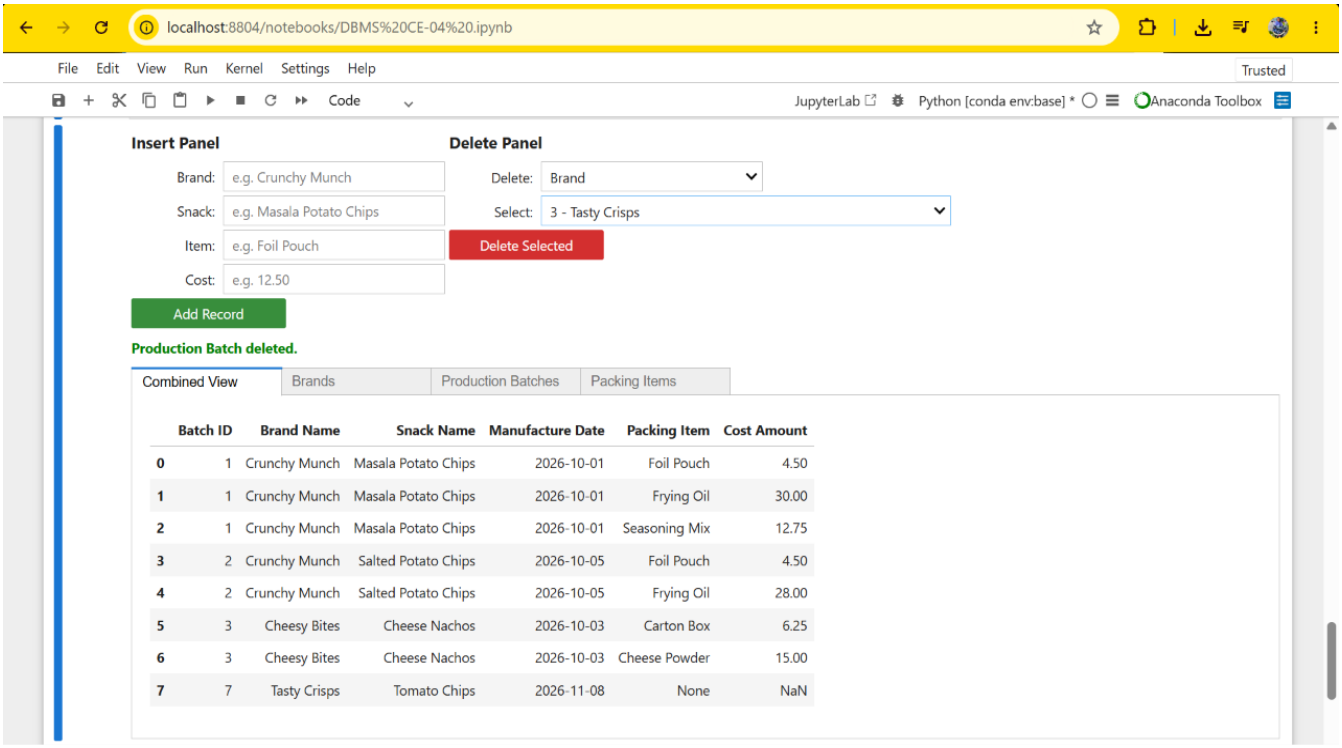


**Figure 17:** The Delete Panel dropdown lists the existing production batches.




| | Batch ID | Brand Name | Snack Name | Manufacture Date | Packing Item | Cost Amount |
|---|---|---|---|---|---|---|
| 0 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Foil Pouch | 4.50 |
| 1 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Frying Oil | 30.00 |
| 2 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Seasoning Mix | 12.75 |
| 3 | 2 | Crunchy Munch | Salted Potato Chips | 2026-10-05 | Foil Pouch | 4.50 |
| 4 | 2 | Crunchy Munch | Salted Potato Chips | 2026-10-05 | Frying Oil | 28.00 |
| 5 | 3 | Cheesy Bites | Cheese Nachos | 2026-10-03 | Carton Box | 6.25 |
| 6 | 3 | Cheesy Bites | Cheese Nachos | 2026-10-03 | Cheese Powder | 15.00 |
| 7 | 6 | Crunchy Munch | Spicy Chips | 2026-11-08 | Label | 3.50 |
| 8 | 7 | Tasty Crisps | Tomato Chips | 2026-11-08 | None | NaN |

**Figure 18:** Before deletion – Batch 6 (Crunchy Munch / Spicy Chips) selected.

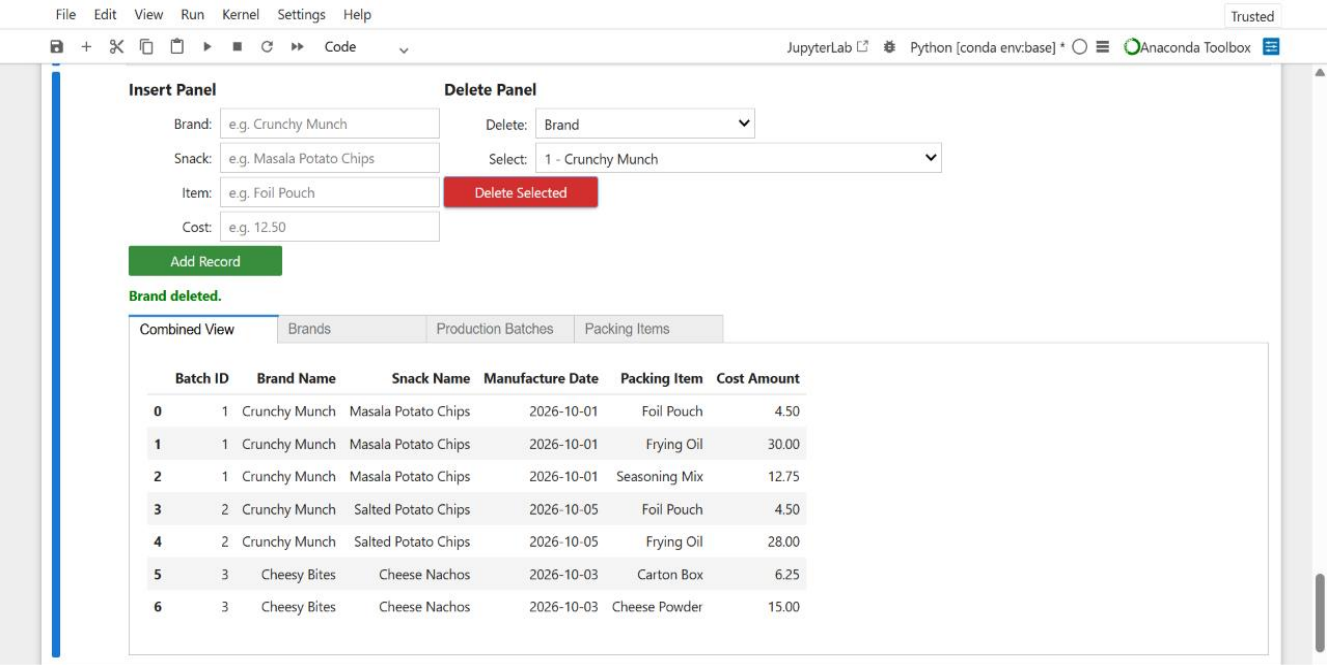



| | Batch ID | Brand Name | Snack Name | Manufacture Date | Packing Item | Cost Amount |
|---|---|---|---|---|---|---|
| 0 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Foil Pouch | 4.50 |
| 1 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Frying Oil | 30.00 |
| 2 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Seasoning Mix | 12.75 |
| 3 | 2 | Crunchy Munch | Salted Potato Chips | 2026-10-05 | Foil Pouch | 4.50 |
| 4 | 2 | Crunchy Munch | Salted Potato Chips | 2026-10-05 | Frying Oil | 28.00 |
| 5 | 3 | Cheesy Bites | Cheese Nachos | 2026-10-03 | Carton Box | 6.25 |
| 6 | 3 | Cheesy Bites | Cheese Nachos | 2026-10-03 | Cheese Powder | 15.00 |
| 7 | 7 | Tasty Crisps | Tomato Chips | 2026-11-08 | None | NaN |

**Figure 19:** After deletion – "Production Batch deleted." Batch 6 and its "Label" item are gone from the Combined View. The dropdown has moved to a valid option (3 – Tasty Crisps selected, before the brand deletion).

## 5.8 Deleting a Brand (cascade to batches and items).

Delete type “Brand” was selected and Brand 3 (Tasty Crisps) was deleted. Its production batch (Batch 7) was automatically removed as well, so the Combined View no longer shows any Tasty Crisps row.




| | Batch ID | Brand Name | Snack Name | Manufacture Date | Packing Item | Cost Amount |
|---|---|---|---|---|---|---|
| 0 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Foil Pouch | 4.50 |
| 1 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Frying Oil | 30.00 |
| 2 | 1 | Crunchy Munch | Masala Potato Chips | 2026-10-01 | Seasoning Mix | 12.75 |
| 3 | 2 | Crunchy Munch | Salted Potato Chips | 2026-10-05 | Foil Pouch | 4.50 |
| 4 | 2 | Crunchy Munch | Salted Potato Chips | 2026-10-05 | Frying Oil | 28.00 |
| 5 | 3 | Cheesy Bites | Cheese Nachos | 2026-10-03 | Carton Box | 6.25 |
| 6 | 3 | Cheesy Bites | Cheese Nachos | 2026-10-03 | Cheese Powder | 15.00 |

**Figure 20:** After deletion – “Brand deleted.” Tasty Crisps and its batch have disappeared from the Combined View.

## 5.9 Dropdowns never point at a deleted ID.

After every deletion, load_targets() rebuilds the dropdown from the live database. The screenshots below show that the deleted brand, batch and items are no longer offered.

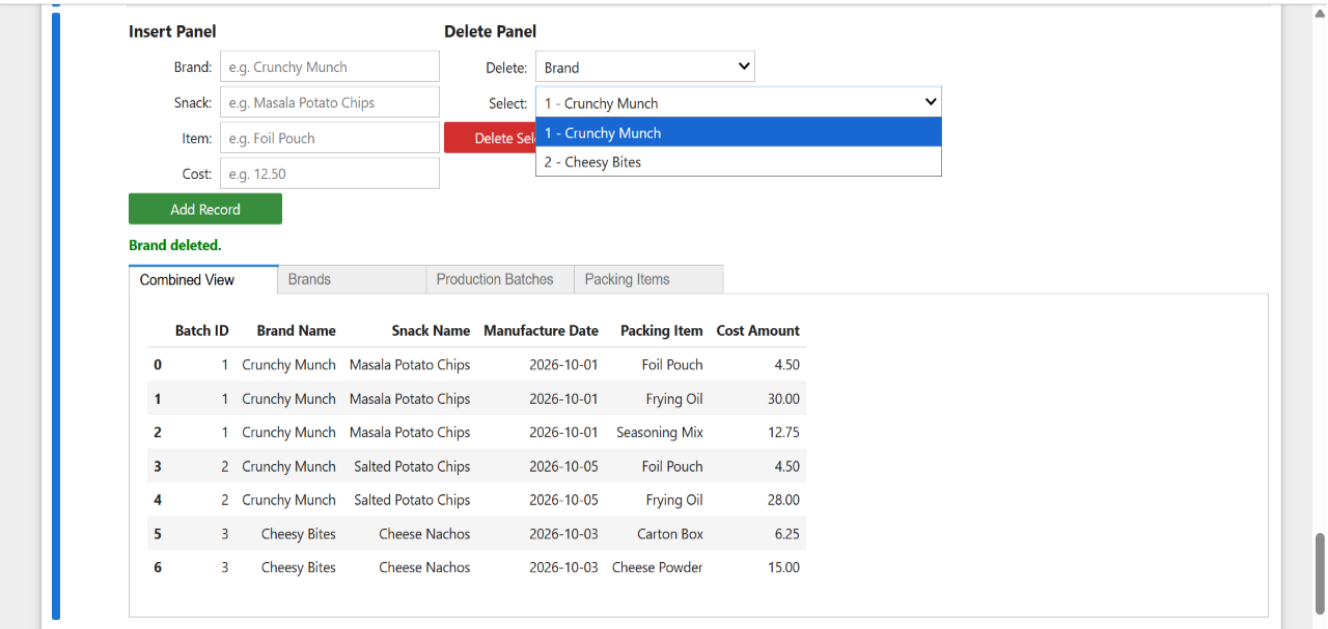



**Figure 21:** Brand dropdown after deletion – only Brand 1 (Crunchy Munch) and Brand 2 (Cheesy Bites) are listed.

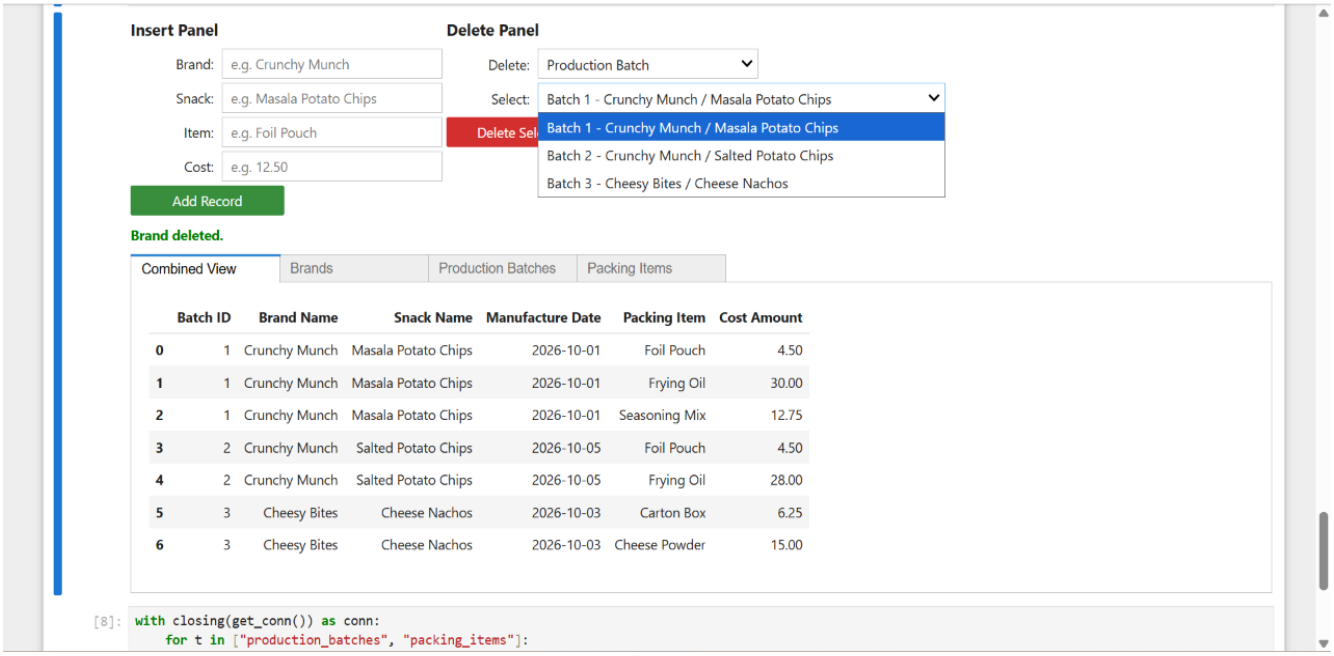



**Figure 22:** Production Batch dropdown after deletion – only Batches 1, 2 and 3 remain.

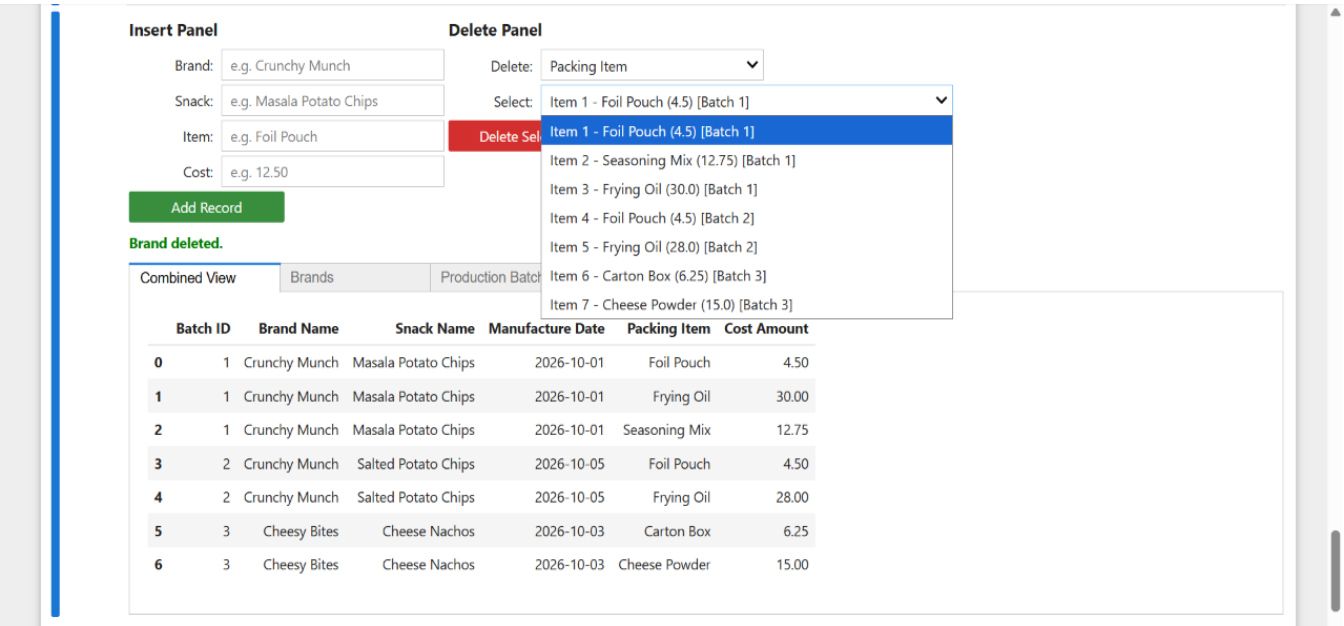



**Figure 23:** Packing Item dropdown after deletion – only Items 1 to 7 remain; the deleted items are not listed.

# 6. Conclusion.

A complete relational Snack Brand & Production Batch Management Dashboard was built with Python, SQLite and ipywidgets, running entirely inside a Jupyter Notebook. The three tables are linked by foreign keys with ON DELETE CASCADE, so deleting a brand or a batch automatically removes all dependent records, while deleting a packing item removes only that single row. Input validation prevents blank, non-numeric and negative entries, and a single refresh routine keeps all four tabs and the delete dropdown synchronised with the database after every action. Testing confirmed that every requirement of the assignment is met.

Through this project I learned how primary and foreign keys enforce data integrity, why PRAGMA foreign_keys = ON is essential in SQLite, how JOINs combine related tables, and how to build an interactive notebook application using ipywidgets.

# Appendix – Complete Source Code.

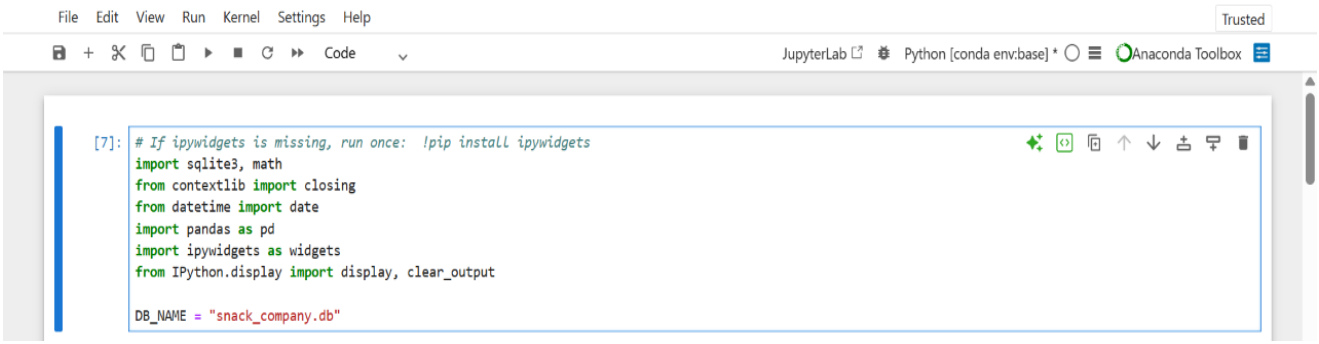
## How to run this project:-

- Open Anaconda / Jupyter Notebook and create a new Python 3 notebook (requires ipywidgets and pandas, both included with Anaconda; if ipywidgets is missing, run !pip install ipywidgets once).
- Copy each code block below (CELL 1 to CELL 4) into its own notebook cell, in order. CELL 5 is an optional foreign-key check.
- Run the cells from top to bottom (Shift + Enter on each cell, or Kernel → Restart Kernel and Run All Cells).
- The app (Insert Panel, Delete Panel and four tabs) appears directly below CELL 4. The database file snack_company.db is created automatically and seeded with sample data on the first run, so the output matches Figure 2.
- To start fresh, delete snack_company.db and run all cells again.

The code below was executed in Jupyter Notebook, one block per cell, in the order shown. The manufacture date of a new batch is set automatically to today's date when a record is added.

```
# ===== CELL 1: Imports =====
import sqlite3, math
from contextlib import closing
from datetime import date
import pandas as pd
import ipywidgets as widgets
from IPython.display import display, clear_output


DB_NAME = "snack_company.db"
```




```
# ===== CELL 2: Database, schema and seed data =====
def get_conn():
    conn = sqlite3.connect(DB_NAME)
    conn.execute("PRAGMA foreign_keys = ON")   # SQLite ignores FKs unless this is on
    return conn


def create_tables():
    with closing(get_conn()) as conn, conn:
        conn.executescript("""
        CREATE TABLE IF NOT EXISTS brands (
            brand_id        INTEGER PRIMARY KEY AUTOINCREMENT,
            brand_name      TEXT NOT NULL UNIQUE,
            supervisor_email TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS production_batches (
            batch_id        INTEGER PRIMARY KEY AUTOINCREMENT,
            brand_id        INTEGER NOT NULL,
            snack_name      TEXT NOT NULL,
            manufacture_date TEXT NOT NULL,
            FOREIGN KEY (brand_id) REFERENCES brands(brand_id) ON DELETE CASCADE
        );
        CREATE TABLE IF NOT EXISTS packing_items (
            item_id         INTEGER PRIMARY KEY AUTOINCREMENT,
            batch_id        INTEGER NOT NULL,
            item_description TEXT NOT NULL,
```

```
        cost_amount      REAL NOT NULL CHECK (cost_amount >= 0),
        FOREIGN KEY (batch_id) REFERENCES production_batches(batch_id) ON DELETE CASCADE
    );
    """)
SAMPLE = [
  ("Crunchy Munch", "crunchy.supervisor@snackco.com", [
    ("Masala Potato Chips", "2026-10-01", [("Foil Pouch", 4.50), ("Seasoning Mix", 12.75), ("Frying Oil", 30.00)]),
    ("Salted Potato Chips", "2026-10-05", [("Foil Pouch", 4.50), ("Frying Oil", 28.00)]),
  ]),
  ("Cheesy Bites", "cheesy.supervisor@snackco.com", [
    ("Cheese Nachos", "2026-10-03", [("Carton Box", 6.25), ("Cheese Powder", 15.00)]),
  ]),
]

def seed_if_empty():
  with closing(get_conn()) as conn, conn:
    if conn.execute("SELECT COUNT(*) FROM brands").fetchone()[0] > 0:
      return
    for brand, email, batches in SAMPLE:
      bid = conn.execute("INSERT INTO brands (brand_name, supervisor_email) VALUES (?,?)",
                         (brand, email)).lastrowid
      for snack, mdate, items in batches:
        batch_id = conn.execute(
          "INSERT INTO production_batches (brand_id, snack_name, manufacture_date) VALUES (?,?,?)",
          (bid, snack, mdate)).lastrowid
        conn.executemany(
          "INSERT INTO packing_items (batch_id, item_description, cost_amount) VALUES (?,?,?)",
          [(batch_id, d, c) for d, c in items])

create_tables()
seed_if_empty()
```

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

[4]:
```
def get_conn():
    conn = sqlite3.connect(DB_NAME)
    conn.execute("PRAGMA foreign_keys = ON")   # SQLite ignores FKs unless this is on
    return conn

def create_tables():
    with closing(get_conn()) as conn, conn:
        conn.executescript("""
        CREATE TABLE IF NOT EXISTS brands (
            brand_id         INTEGER PRIMARY KEY AUTOINCREMENT,
            brand_name       TEXT NOT NULL UNIQUE,
            supervisor_email TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS production_batches (
            batch_id         INTEGER PRIMARY KEY AUTOINCREMENT,
            brand_id         INTEGER NOT NULL,
            snack_name       TEXT NOT NULL,
            manufacture_date TEXT NOT NULL,
            FOREIGN KEY (brand_id) REFERENCES brands(brand_id) ON DELETE CASCADE
        );
        CREATE TABLE IF NOT EXISTS packing_items (
            item_id          INTEGER PRIMARY KEY AUTOINCREMENT,
            batch_id         INTEGER NOT NULL,
            item_description TEXT NOT NULL,
            cost_amount      REAL NOT NULL CHECK (cost_amount >= 0),
            FOREIGN KEY (batch_id) REFERENCES production_batches(batch_id) ON DELETE CASCADE
        );
        """)
```

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
SAMPLE = [
    ("Crunchy Munch", "crunchy.supervisor@snackco.com", [
        ("Masala Potato Chips", "2026-10-01", [("Foil Pouch", 4.50), ("Seasoning Mix", 12.75), ("Frying Oil", 30.00)]),
        ("Salted Potato Chips", "2026-10-05", [("Foil Pouch", 4.50), ("Frying Oil", 28.00)]),
    ]),
    ("Cheesy Bites", "cheesy.supervisor@snackco.com", [
        ("Cheese Nachos", "2026-10-03", [("Carton Box", 6.25), ("Cheese Powder", 15.00)]),
    ]),
]

def seed_if_empty():
    with closing(get_conn()) as conn, conn:
        if conn.execute("SELECT COUNT(*) FROM brands").fetchone()[0] > 0:
            return
        for brand, email, batches in SAMPLE:
            bid = conn.execute("INSERT INTO brands (brand_name, supervisor_email) VALUES (?,?)",
                               (brand, email)).lastrowid
            for snack, mdate, items in batches:
                batch_id = conn.execute(
                    "INSERT INTO production_batches (brand_id, snack_name, manufacture_date) VALUES (?,?,?)",
                    (bid, snack, mdate)).lastrowid
                conn.executemany(
                    "INSERT INTO packing_items (batch_id, item_description, cost_amount) VALUES (?,?,?)",
                    [(batch_id, d, c) for d, c in items])

create_tables()
seed_if_empty()
```

```
# ===== CELL 3: Logic (queries, add, delete, validation) =====
```

```
def query_rows(sql):
    with closing(get_conn()) as conn:
        return conn.execute(sql).fetchall()

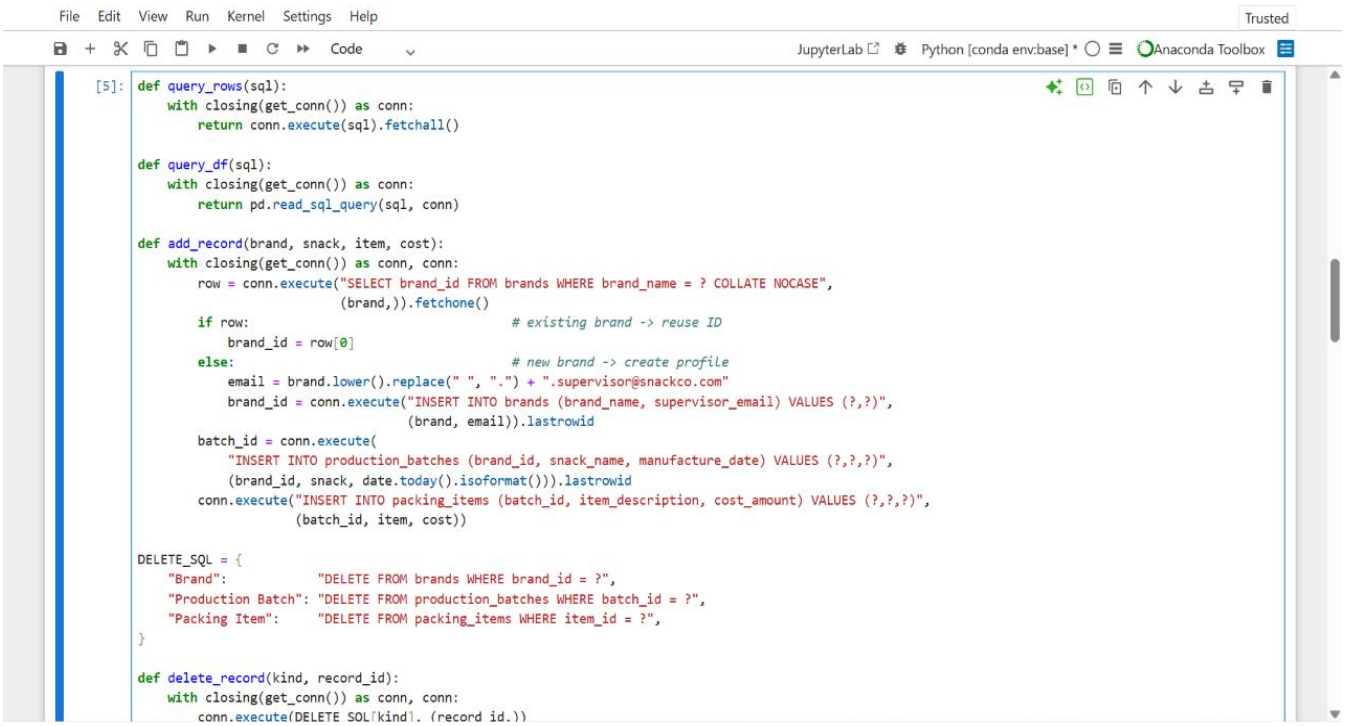

def query_df(sql):
    with closing(get_conn()) as conn:
        return pd.read_sql_query(sql, conn)
```

```
def add_record(brand, snack, item, cost):
    with closing(get_conn()) as conn, conn:
        row = conn.execute("SELECT brand_id FROM brands WHERE brand_name = ? COLLATE NOCASE",
                           (brand,)).fetchone()
        if row:                             # existing brand -> reuse ID
            brand_id = row[0]
        else:                               # new brand -> create profile
            email = brand.lower().replace(" ", ".") + ".supervisor@snackco.com"
            brand_id = conn.execute("INSERT INTO brands (brand_name, supervisor_email) VALUES (?,?)",
                                    (brand, email)).lastrowid
        batch_id = conn.execute(
            "INSERT INTO production_batches (brand_id, snack_name, manufacture_date) VALUES (?,?,?)",
            (brand_id, snack, date.today().isoformat())).lastrowid
        conn.execute("INSERT INTO packing_items (batch_id, item_description, cost_amount) VALUES (?,?,?)",
                     (batch_id, item, cost))

DELETE_SQL = {
    "Brand":           "DELETE FROM brands WHERE brand_id = ?",
    "Production Batch": "DELETE FROM production_batches WHERE batch_id = ?",
    "Packing Item":    "DELETE FROM packing_items WHERE item_id = ?",
}

def delete_record(kind, record_id):
    with closing(get_conn()) as conn, conn:
        conn.execute(DELETE_SQL[kind], (record_id,))
def parse_cost(text):
    try:
        v = float(text.strip())
    except ValueError:
```

```
        return None
    return v if math.isfinite(v) and v >= 0 else None
JOIN_SQL = """
SELECT b.batch_id AS "Batch ID", br.brand_name AS "Brand Name", b.snack_name AS "Snack Name",
       b.manufacture_date AS "Manufacture Date", p.item_description AS "Packing Item",
       p.cost_amount AS "Cost Amount"
FROM production_batches b
JOIN brands br ON br.brand_id = b.brand_id
LEFT JOIN packing_items p ON p.batch_id = b.batch_id
ORDER BY b.batch_id
"""
```






```
# ===== CELL 4: The GUI =====
```

```
# ---------- Insert panel ----------

brand_in = widgets.Text(description="Brand:",  placeholder="e.g. Crunchy Munch")

snack_in = widgets.Text(description="Snack:",  placeholder="e.g. Masala Potato Chips")

item_in  = widgets.Text(description="Item:",   placeholder="e.g. Foil Pouch")

cost_in  = widgets.Text(description="Cost:",   placeholder="e.g. 12.50")

add_btn  = widgets.Button(description="Add Record", button_style="success")


# ---------- Delete panel ----------

delete_type = widgets.Dropdown(options=["Brand", "Production Batch", "Packing Item"], description="Delete:")

target_dd   = widgets.Dropdown(description="Select:", layout=widgets.Layout(width="480px"))

del_btn     = widgets.Button(description="Delete Selected", button_style="danger")


status = widgets.HTML("")


# ---------- Tabs ----------

outs = [widgets.Output() for _ in range(4)]

tabs = widgets.Tab(children=outs)

for i, t in enumerate(["Combined View", "Brands", "Production Batches", "Packing Items"]):

    tabs.set_title(i, t)
```

```
def msg(text, ok=True):
    status.value = f"<b style='color:{'green' if ok else 'red'}'>{text}</b>"


def refresh_tables():
    queries = [JOIN_SQL, "SELECT * FROM brands", "SELECT * FROM production_batches",
               "SELECT * FROM packing_items"]
    for out, q in zip(outs, queries):
        df = query_df(q)
        with out:
            clear_output(wait=True)
            display(df)


def load_targets():
    kind = delete_type.value
    if kind == "Brand":
        rows = query_rows("SELECT brand_id, brand_name FROM brands ORDER BY brand_id")
        opts = [(f"{r[0]} - {r[1]}", r[0]) for r in rows]
    elif kind == "Production Batch":
        rows = query_rows("""SELECT b.batch_id, br.brand_name, b.snack_name FROM production_batches b
                             JOIN brands br ON br.brand_id = b.brand_id ORDER BY b.batch_id""")
        opts = [(f"Batch {r[0]} - {r[1]} / {r[2]}", r[0]) for r in rows]
    else:
        rows = query_rows("SELECT item_id, item_description, cost_amount, batch_id FROM packing_items
ORDER BY item_id")
        opts = [(f"Item {r[0]} - {r[1]} ({r[2]}) [Batch {r[3]}]", r[0]) for r in rows]
    target_dd.options = opts
    target_dd.value = opts[0][1] if opts else None   # never points at a deleted ID
```

```
def refresh_all():
    refresh_tables()
    load_targets()

def on_add(_):
    brand, snack, item = brand_in.value.strip(), snack_in.value.strip(), item_in.value.strip()
    if not brand or not snack or not item or not cost_in.value.strip():
        return msg("All fields are required.", False)
    cost = parse_cost(cost_in.value)
    if cost is None:
        return msg("Cost must be a non-negative number.", False)
    add_record(brand, snack, item, cost)
    brand_in.value = snack_in.value = item_in.value = cost_in.value = ""
    refresh_all()
    msg("Record added successfully.")

def on_delete(_):
    if target_dd.value is None:
        return msg("Nothing to delete.", False)
    kind = delete_type.value
    delete_record(kind, target_dd.value)
    refresh_all()
    msg(f"{kind} deleted.")

add_btn.on_click(on_add)
del_btn.on_click(on_delete)
delete_type.observe(lambda change: load_targets(), names="value")
```

```
ui = widgets.VBox([
    widgets.HBox([
        widgets.VBox([widgets.HTML("<h3>Insert Panel</h3>"), brand_in, snack_in, item_in, cost_in, add_btn]),
        widgets.VBox([widgets.HTML("<h3>Delete Panel</h3>"), delete_type, target_dd, del_btn]),
    ]),
    status,
    tabs,
])

refresh_all()
display(ui)
# ===== CELL 5: Foreign key check (for the report) =====
with closing(get_conn()) as conn:
    for t in ["production_batches", "packing_items"]:
        print(t)
        for row in conn.execute(f"PRAGMA foreign_key_list({t})"):
            print("  references:", row[2], "| on_delete:", row[6])
```

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab

Python [conda env:base] *

Anaconda Toolbox

[6]:

```
# ---------- Insert panel ----------
brand_in = widgets.Text(description="Brand:",  placeholder="e.g. Crunchy Munch")
snack_in = widgets.Text(description="Snack:",  placeholder="e.g. Masala Potato Chips")
item_in  = widgets.Text(description="Item:",   placeholder="e.g. Foil Pouch")
cost_in  = widgets.Text(description="Cost:",   placeholder="e.g. 12.50")
add_btn  = widgets.Button(description="Add Record", button_style="success")

# ---------- Delete panel ----------
delete_type = widgets.Dropdown(options=["Brand", "Production Batch", "Packing Item"], description="Delete:")
target_dd   = widgets.Dropdown(description="Select:", layout=widgets.Layout(width="480px"))
del_btn     = widgets.Button(description="Delete Selected", button_style="danger")

status = widgets.HTML("")

# ---------- Tabs ----------
outs = [widgets.Output() for _ in range(4)]
tabs = widgets.Tab(children=outs)
for i, t in enumerate(["Combined View", "Brands", "Production Batches", "Packing Items"]):
    tabs.set_title(i, t)

def msg(text, ok=True):
    status.value = f"<b style='color:{'green' if ok else 'red'}'>{text}</b>"

def refresh_tables():
    queries = [JOIN_SQL, "SELECT * FROM brands", "SELECT * FROM production_batches",
               "SELECT * FROM packing_items"]
    for out, q in zip(outs, queries):
        df = query_df(q)
        with out:
            clear_output(wait=True)
            display(df)
```

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab

Python [conda env:base] *

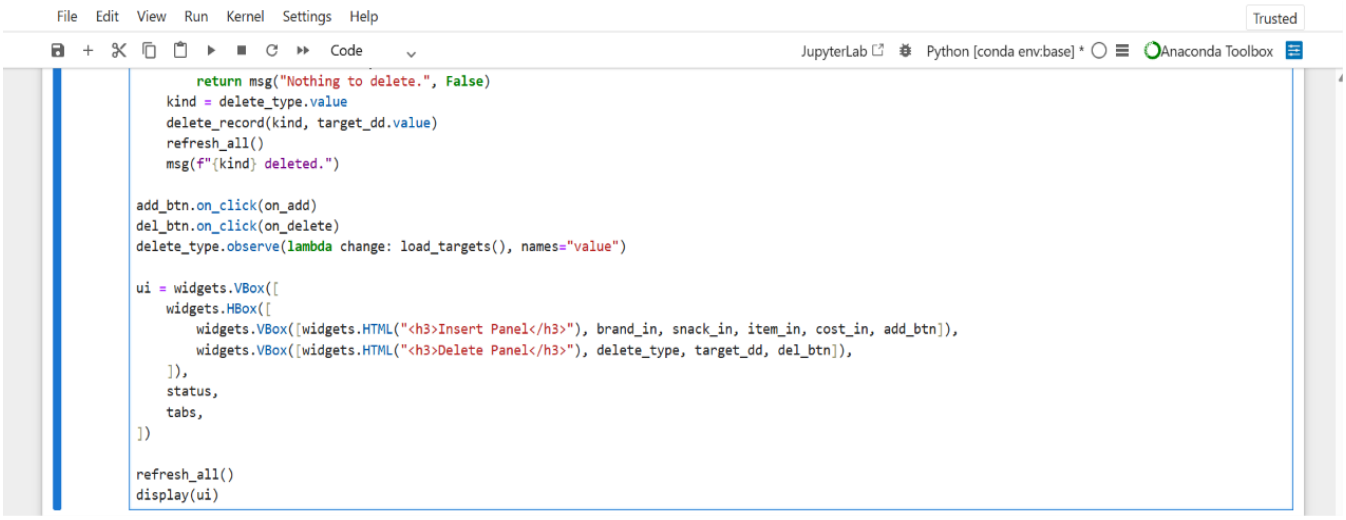
Anaconda Toolbox

```
def load_targets():
    kind = delete_type.value
    if kind == "Brand":
        rows = query_rows("SELECT brand_id, brand_name FROM brands ORDER BY brand_id")
        opts = [(f"{r[0]} - {r[1]}", r[0]) for r in rows]
    elif kind == "Production Batch":
        rows = query_rows("""SELECT b.batch_id, br.brand_name, b.snack_name FROM production_batches b
                             JOIN brands br ON br.brand_id = b.brand_id ORDER BY b.batch_id""")
        opts = [(f"Batch {r[0]} - {r[1]} / {r[2]}", r[0]) for r in rows]
    else:
        rows = query_rows("SELECT item_id, item_description, cost_amount, batch_id FROM packing_items ORDER BY item_id")
        opts = [(f"Item {r[0]} - {r[1]} ({r[2]}) [Batch {r[3]}]", r[0]) for r in rows]
    target_dd.options = opts
    target_dd.value = opts[0][1] if opts else None   # never points at a deleted ID

def refresh_all():
    refresh_tables()
    load_targets()

def on_add(_):
    brand, snack, item = brand_in.value.strip(), snack_in.value.strip(), item_in.value.strip()
    if not brand or not snack or not item or not cost_in.value.strip():
        return msg("All fields are required.", False)
    cost = parse_cost(cost_in.value)
    if cost is None:
        return msg("Cost must be a non-negative number.", False)
    add_record(brand, snack, item, cost)
    brand_in.value = snack_in.value = item_in.value = cost_in.value = ""
    refresh_all()
    msg("Record added successfully.")

def on_delete(_):
    if target_dd.value is None:
```


File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python [conda env:base] *
Anaconda Toolbox
return msg("Nothing to delete.", False)
kind = delete_type.value
delete_record(kind, target_dd.value)
refresh_all()
msg(f"{kind} deleted.")
add_btn.on_click(on_add)
del_btn.on_click(on_delete)
delete_type.observe(lambda change: load_targets(), names="value")
ui = widgets.VBox([
widgets.HBox([
widgets.VBox([widgets.HTML("<h3>Insert Panel</h3>"), brand_in, snack_in, item_in, cost_in, add_btn]),
widgets.VBox([widgets.HTML("<h3>Delete Panel</h3>"), delete_type, target_dd, del_btn]),
]),
status,
tabs,
])
refresh_all()
display(ui)